A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

William Zijie Zhang

Master's Thesis Defence
October 2nd

Research director: Antoine Lesage-Landry
Department of Electrical Engineering, Polytechnique Montréal

Outline

Introduction

Example

Single canonicalization

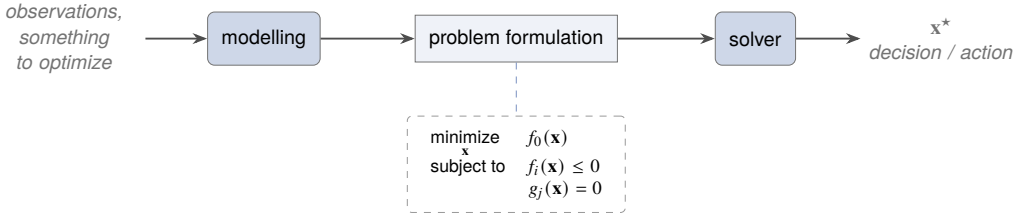
Parametric canonicalization

Conclusion

Optimization in practice



Optimization in practice



Convex optimization

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f_0(\mathbf{x}) \\ \text{subject to} \quad & f_i(\mathbf{x}) \leq 0 \\ & g_j(\mathbf{x}) = 0 \end{aligned}$$

- variable $\mathbf{x} \in \mathbb{R}^n$
- the equality constraints g_j are affine, and f_0 and the f_i are convex
- why care about convexity?

Boyd and Vandenberghe (2004).

Convex optimization

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f_0(\mathbf{x}) \\ \text{subject to} \quad & f_i(\mathbf{x}) \leq 0 \\ & g_j(\mathbf{x}) = 0 \end{aligned}$$

- variable $\mathbf{x} \in \mathbb{R}^n$
- the equality constraints g_j are affine, and f_0 and the f_i are convex
- why care about convexity?
  - classic answer: a local optimum is a global optimum
  - modern answer: very actionable, because of reliable software

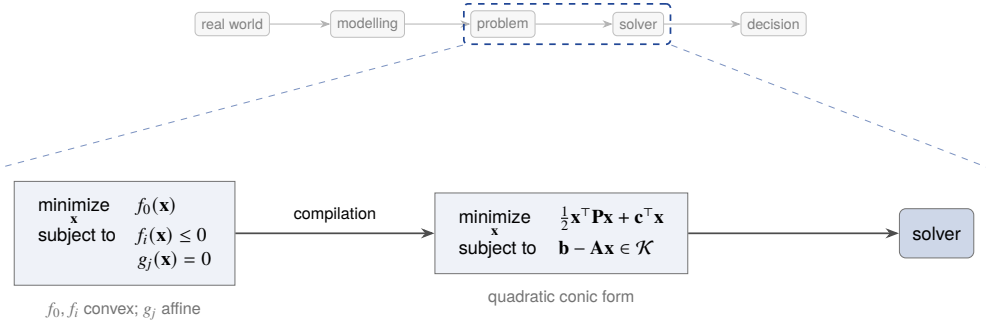
Boyd and Vandenberghe (2004).

Conic form

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & \frac{1}{2}\mathbf{x}^\top\mathbf{P}\mathbf{x} + \mathbf{c}^\top\mathbf{x} \\ \text{subject to} & \mathbf{b} - \mathbf{A}\mathbf{x} \in \mathcal{K} \end{array}$$

- **problem data**: four arrays $\mathbf{P} \in \mathbb{S}^n_+$, $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c} \in \mathbb{R}^n$
- $\mathcal{K} \subseteq \mathbb{R}^m$ is a product of standard cones
- when $\mathcal{K} = \mathbb{R}^m_+$ is the nonnegative orthant, this is a QP

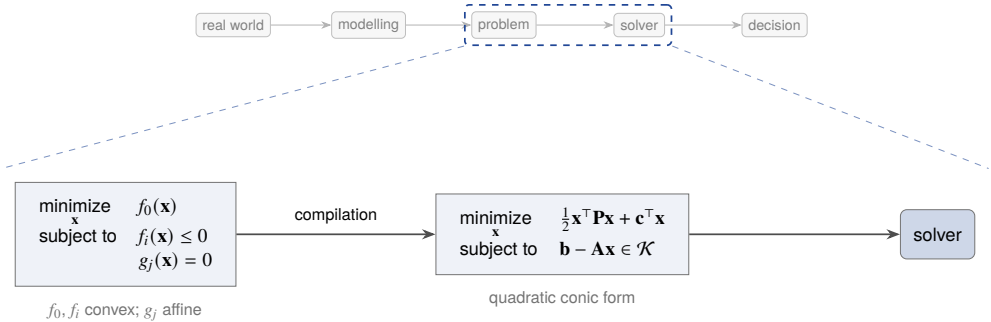
From convex problem to solver



algorithms

interior point simplex (LP)
operator splitting active set (QP)

From convex problem to solver



algorithms

interior point simplex (LP)
operator splitting active set (QP)

solvers

commercial: Gurobi, MOSEK, COPT
open source: OSQP, Clarabel, SCS

Disciplined convex programming

- a grammar for specifying convex optimization problems
- convex and concave functions are built by combining a set of **atoms**
- each atom has known curvature, monotonicity in each argument, and sign
- simple rules specify how atoms can be composed

Grant, Boyd and Ye (2006).

Disciplined convex programming

- a grammar for specifying convex optimization problems
- convex and concave functions are built by combining a set of **atoms**
- each atom has known curvature, monotonicity in each argument, and sign
- simple rules specify how atoms can be composed

- a problem that follows the DCP rules is convex by construction
- and can be solved globally and efficiently — in theory and in practice

Grant, Boyd and Ye (2006).

DCP-based modelling languages

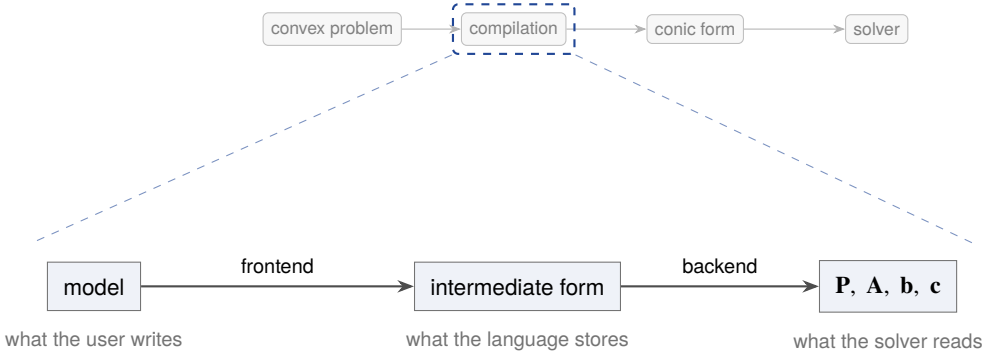
- `CVX` (MATLAB), `CVXPY` (Python), `Convex.jl` (Julia), `CVXR` (R), `YALMIP` (MATLAB)
- `AMPL`, `GAMS` and `JuMP` do not implement DCP

DCP-based modelling languages

- `CVX` (MATLAB), `CVXPY` (Python), `Convex.jl` (Julia), `CVXR` (R), `YALMIP` (MATLAB)
- `AMPL`, `GAMS` and `JuMP` do not implement DCP
- the two tasks of a DCP-based modelling language:
  - **analysis**: verify that a high-level problem description follows the DCP rules
  - **synthesis**: build an equivalent problem in conic form, called canonicalization

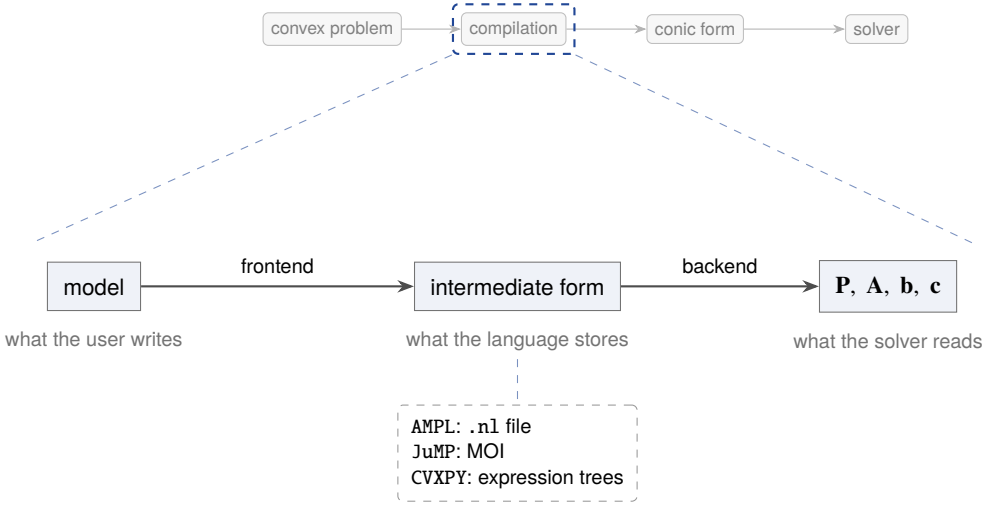
Compilation

In DCP-based languages this step is called canonicalization.

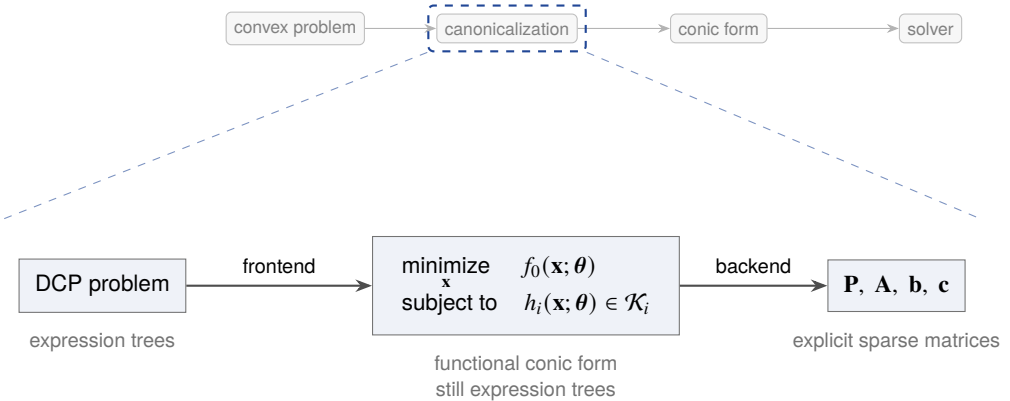


Compilation

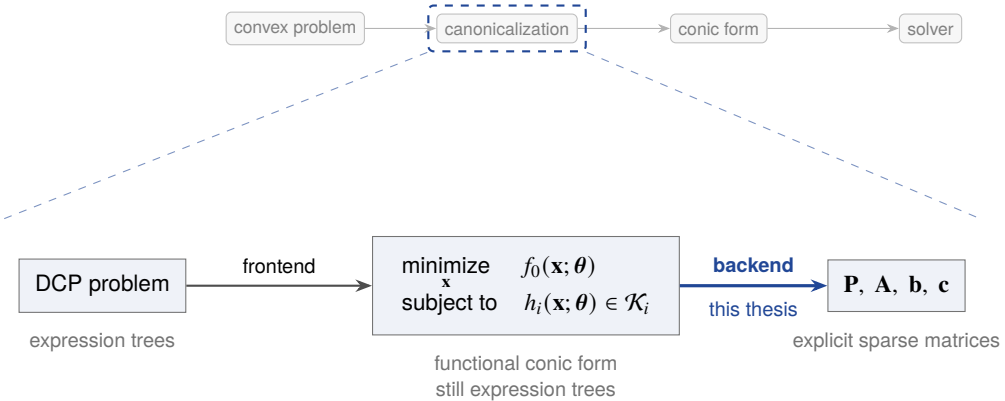
In DCP-based languages this step is called canonicalization.



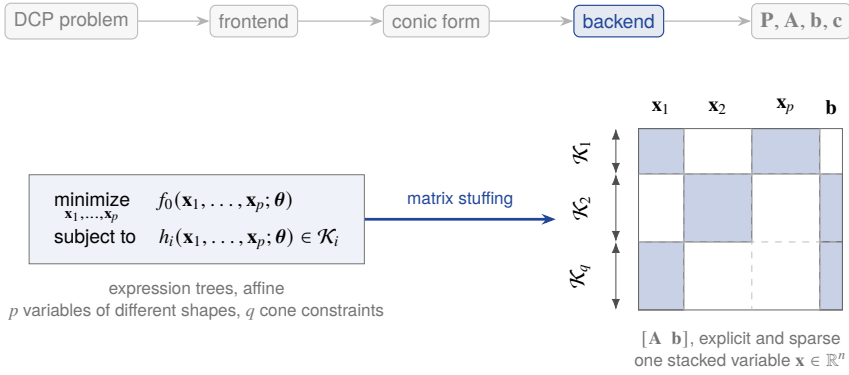
Canonicalization: frontend and backend



Canonicalization: frontend and backend

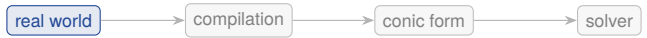


Problem statement



- most of the compilation time is spent in this one step
- the frontend takes care of $\mathcal{K}$
- the goal of this thesis is to reduce CVXPY's compilation time

Fitting with regularization



$$\begin{array}{ll} \text{minimize} & \|\mathbf{F}\mathbf{x} - \mathbf{g}\|_2 + \mu\|\mathbf{x}\|_2 \\ \text{subject to} & \mathbf{x} \geq \mathbf{0} \end{array}$$

- with variable $\mathbf{x} \in \mathbb{R}^n$, data $\mathbf{F} \in \mathbb{R}^{m\times n}$, parameters $\mathbf{g} \in \mathbb{R}^m$ and $\mu \geq 0$
- interpretation: find a good approximation $\mathbf{F}\mathbf{x} \approx \mathbf{g}$ with small $\mathbf{x}$, and μ sets the trade-off between the two
- the parameters change from solve to solve; $\mathbf{F}$ does not — one problem family, and each value of the parameters gives a problem instance

Convex optimization



$$\underset{\mathbf{x}\geq\mathbf{0}}{\text{minimize}} \quad \underbrace{\|\mathbf{F}\mathbf{x}-\mathbf{g}\|_2}_{\text{convex}} + \underbrace{\mu\|\mathbf{x}\|_2}_{\text{convex}}$$

- $\|\cdot\|_2$ is convex, and $\mathbf{F}\mathbf{x}-\mathbf{g}$ is affine in $\mathbf{x}$, so the composition $\|\mathbf{F}\mathbf{x}-\mathbf{g}\|_2$ is convex
- $\mu \geq 0$, so $\mu\|\mathbf{x}\|_2$ is a nonnegative scaling of a convex function
- a sum of convex functions is convex, and the constraint $\mathbf{x} \geq \mathbf{0}$ is affine

A DCP-based modelling language




```
import cvxpy as cp
import numpy as np

F  = np.random.randn(m, n)
x  = cp.Variable(n)
g  = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)

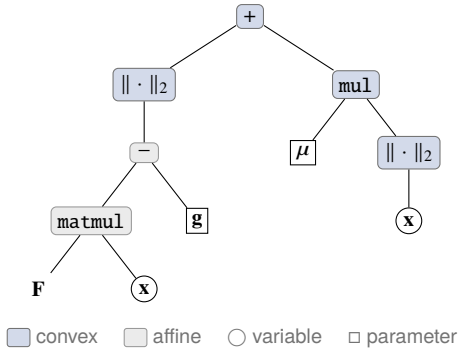
objective = cp.Minimize(
    cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

- the problem is written the way it is stated: atoms and operators, not matrices
- `Parameter` marks what changes between solves; `Variable` marks what is solved for
- nothing here says how to solve it, and nothing says it is convex

Disciplined convex programming



CVXPY stores the objective and each constraint as an expression tree:



Canonicalization: the frontend



$$\begin{aligned} \underset{t_1,\, t_2,\, \mathbf{x}}{\text{minimize}} \quad & t_1 + \mu t_2 \\ \text{subject to} \quad & \mathbf{x} \geq \mathbf{0} \\ & (t_1,\ \mathbf{F}\mathbf{x} - \mathbf{g}) \in \mathcal{K}_{\text{soc},\, m+1} \\ & (t_2,\ \mathbf{x}) \in \mathcal{K}_{\text{soc},\, n+1} \end{aligned}$$

- the norms become epigraph variables t_1, t_2; the cone is the product $\mathcal{K} = \mathbb{R}^n_+ \times \mathcal{K}_{\text{soc},m+1} \times \mathcal{K}_{\text{soc},n+1}$
- every constraint is a cone membership, and every expression left is affine in $(t_1, t_2, \mathbf{x})$
- one affine expression tree per constraint

Canonicalization: the backend



Stack the variables as $\mathbf{z} = (t_1, t_2, \mathbf{x})$ and the data falls out:

$$\mathbf{c}^\top = \begin{matrix} t_1 & t_2 & \mathbf{x} \end{matrix} \begin{bmatrix} 1 & \mu & 0 \end{bmatrix} \qquad [\mathbf{A}\ \ \mathbf{b}] = \begin{matrix} t_1 & t_2 & \mathbf{x} & \mathbf{b} \end{matrix} \begin{bmatrix} 0 & 0 & -\mathbf{I} & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & -\mathbf{F} & -\mathbf{g} \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -\mathbf{I} & 0 \end{bmatrix}$$

- the five row blocks are the three cone constraints, in the order $\mathbb{R}^n_+$, $\mathcal{K}_{\text{soc},m+1}$, $\mathcal{K}_{\text{soc},n+1}$
- the coefficients are negated because the standard form writes $\mathbf{b} - \mathbf{A}\mathbf{x} \in \mathcal{K}$
- $\mathbf{P} = \mathbf{0}$ here: the objective canonicalized to $t_1 + \mu t_2$, which is linear — a quadratic objective would fill it

Canonicalization: the backend

convex problem → compilation → **conic form** → solver

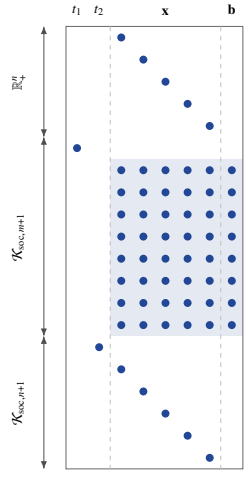
The parameters are $\mathbf{g}$ and μ. Everything else is fixed for the whole problem family:

$$\mathbf{c}^\top = \begin{matrix} t_1 & t_2 & \mathbf{x} \end{matrix} \begin{bmatrix} 1 & \mu & 0 \end{bmatrix} \qquad [\mathbf{A}\ \ \mathbf{b}] = \begin{matrix} t_1 & t_2 & \mathbf{x} & \mathbf{b} \end{matrix} \begin{bmatrix} 0 & 0 & -\mathbf{I} & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & -\mathbf{F} & -\mathbf{g} \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -\mathbf{I} & 0 \end{bmatrix}$$

- μ lands in one entry of $\mathbf{c}$; $\mathbf{g}$ lands in one block of $\mathbf{b}$
- every other entry — the signs, the identities, the data $\mathbf{F}$ — is the same at every solve
- and the *positions* never move at all

Structure in the problem data

convex problem → compilation → conic form → solver

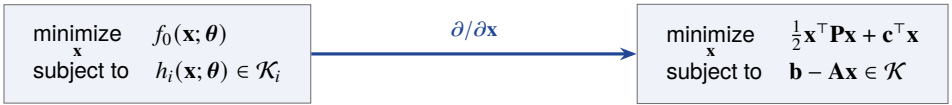


$[\mathbf{A} \quad \mathbf{b}]$ for the example, at $n = 5$ and $m = 8$

- a dense block, two identity diagonals, two lone entries
- none of it was discovered: every block is the Jacobian of the atom that produced it, known before any number is computed

A generic sparse format stores all of it as unrelated (row, column, value) triples.

Canonicalization as sparse differentiation

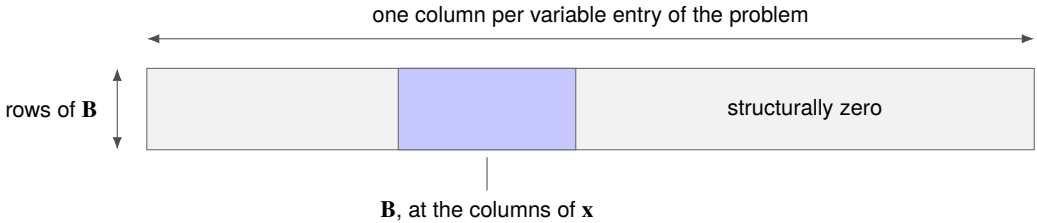


$$\mathbf{A} = \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \qquad \mathbf{P} = \frac{\partial^2 f_0}{\partial \mathbf{x}^2} \qquad \mathbf{b} = \mathbf{h}(\mathbf{0}) \qquad \mathbf{c} = \nabla f_0(\mathbf{0})$$

One evaluation at the origin recovers $\mathbf{b}$ and $\mathbf{c}$.

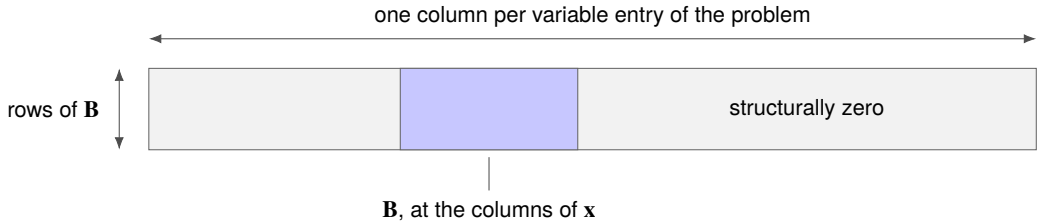
- one Jacobian per node, with respect to *all* the variables, so the chain rule is a single matrix product — not a dictionary of blocks keyed by variable
- the structure comes from the expression: no sparsity detection, no graph colouring
- expressions stay vectorized, never scalarized into lists of scalar coefficients

A dense block at permuted positions



The Jacobian of $\mathbf{Bx}$ for dense data $\mathbf{B}$. A generic sparse format stores one column index *per entry* of the block.

A dense block at permuted positions



The Jacobian of $\mathbf{Bx}$ for dense data $\mathbf{B}$. A generic sparse format stores one column index *per entry* of the block.

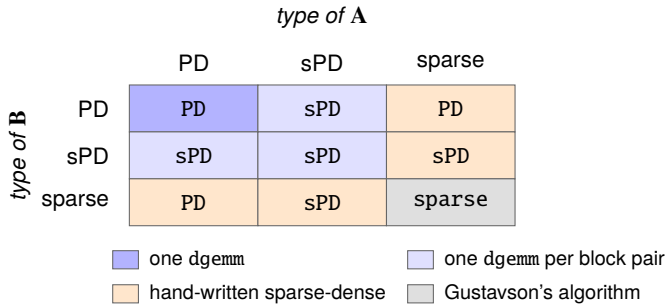
$$\mathbf{A} = \mathbf{S}_r^{\top}\mathbf{X}\mathbf{S}_c, \qquad A_{r_i,\,c_j} = X_{ij}$$

- store the triple $(\mathbf{X}, \mathbf{r}, \mathbf{c})$: one index per row and per column, and the global matrix is never formed
- the storage does not depend on the global dimensions m and n
- $\mathbf{BV}$ gives one such block per column of $\mathbf{V}$, at disjoint rows — kept as a vertical union, with the data stored once

Closed under the operations we need

$$\mathbf{B}^\top \mathbf{A} = \mathbf{S}_{c_B}^\top \mathbf{X}_B^\top \underbrace{\left(\mathbf{S}_{r_B} \mathbf{S}_{r_A}^\top\right)}_{\text{index-set intersection}} \mathbf{X}_A \mathbf{S}_{c_A}$$

The symbolic pass intersects the two sorted index vectors. The numeric pass multiplies the two blocks with one `dgemm`. No selector matrix is ever formed.



Eight of the nine kernels come out dense; cost depends on block size, not on m and n.

First canonicalization

25 problems of CVXPY's public benchmark suite • `get_problem_data` with `Clarabel`, solver excluded • the `ignore_dpp` path • single-threaded, 20-minute limit

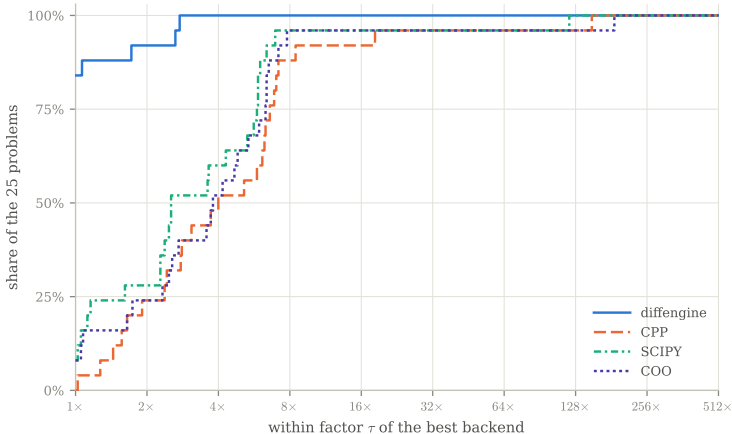
Benchmark	CPP (s)	SCIPY (s)	COO (s)	diffengine (s)	Ratio
UnconstrainedQP	10.32	8.52	13.54	**0.07**	0.01
LeastSquares	9.61	10.08	10.68	**1.45**	0.15
SimpleLP	37.57	29.99	30.96	**6.12**	0.20
Cajas	22.67	2.91	2.89	**1.07**	0.37
ConeMatrixStuffing	11.06	9.34	16.99	**3.95**	0.42
	⋮	16 further problems	⋮		
CVaR	35.85	**24.79**	26.86	24.94	1.01
OptimalAdvertising	6.43	2.32	**2.25**	2.36	1.05
SDPSegfault1132	64.84	**9.51**	43.00	13.21	1.39
ConvexPlasticity	**0.35**	**0.35**	**0.35**	0.85	2.48
Geometric mean					**0.36**

Fastest of the four on 21 of the 25 problems, and more than twice as fast as the best CVXPY backend on 17.

CVaR and OptimalAdvertising are ties; on SDPSegfault1132 and ConvexPlasticity, processing dense constant data dominates.

Performance profile

Share of the 25 problems on which a backend runs within a factor τ of the fastest backend on that problem:

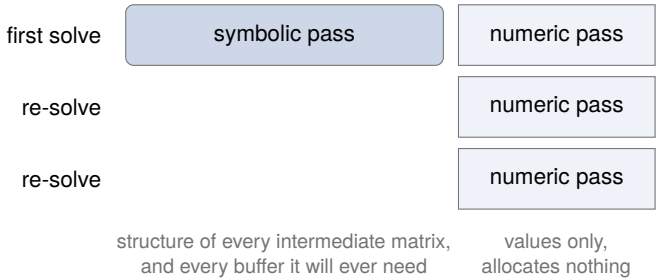


`diffengine` is within $3\times$ of the best backend on *every* problem.

No `CVXPY` backend is fastest on more than two problems, and each needs $\tau \approx 8$ to cover 92–96% of them.

Two passes

The sparsity patterns are fixed across a problem family: a parameter changes the *values* of the problem data, never its *structure*.

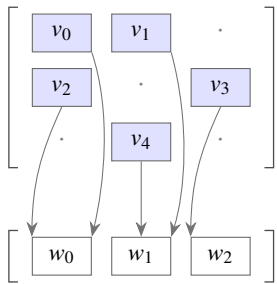


- re-solving a parametrized problem costs one numeric pass
- no restriction on how the parameters enter the problem
- the split itself is borrowed from the interfaces of nonlinear programming solvers

What the symbolic pass stores

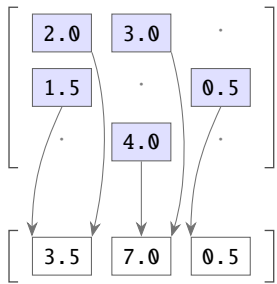
Depending on the kernel, the stored structure is a **sparsity pattern**, a **block layout**, or an **index map**.

symbolic pass, once



$\texttt{map} = (0, 1, 0, 2, 1)$

numeric pass, per evaluation



$\texttt{out}[\texttt{map}[j]]$ += $\texttt{vals}[j]$

The `sum` atom: the symbolic pass records where each nonzero lands, the numeric pass follows the fixed arrows.

`index`, `transpose` and `broadcast` build the same map, with a copy in place of an addition.

The parameter maps

Under DPP the problem data is affine in the parameters, so the whole family collapses to two cached maps:

$$\mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}}, \qquad \operatorname{vec}([\mathbf{A}\ \mathbf{b}]) = \mathbf{R}\,\tilde{\boldsymbol{\theta}}, \qquad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} \boldsymbol{\theta} \\ 1 \end{bmatrix}$$

A re-solve is then one sparse matrix-vector product per map.

`CVXPY`'s three backends are three representations of that same tensor:

- `cvxcore` (C++) — one sparse matrix per parameter slice, looped over at every operator
- `SCIPY` — the slices stacked vertically into one strided two-dimensional sparse matrix
- `COO` — the tensor is never formed: $(\text{row}, \text{column}, \text{value})$ triples extended with a parameter index

The benchmarks that follow compare against all three.

Parametric problems

The seven DPP parametric problems. Each `CVXPY` backend builds the tensor; `diffengine` runs its symbolic pass and one numeric pass. A dash is a tensor that did not finish inside the time limit.

	first canonicalization				re-solve		
Benchmark	`CPP`	`SCIPY`	`C00`	`diffengine`	tensor	`diffengine`	Ratio
ParamSmallMatrixStuffing	7.78	29.06	0.40	**0.21**	0.005	**0.001**	0.29
ParamConeMatrixStuffing	3.16	7.63	0.26	**0.14**	0.003	**0.001**	0.29
SimpleFullyParametrizedLP	—	—	2.17	**0.54**	0.261	**0.126**	0.48
ParametrizedQP	—	—	10.87	**2.53**	1.485	**0.985**	0.66
SimpleScalarParametrizedLP	4.68	4.15	4.48	**1.03**	0.304	**0.247**	0.81
SVMWithL1Regularization	11.00	8.21	9.62	**1.86**	**0.252**	0.496	1.97
FactorCovarianceModel	4.92	4.15	4.66	**0.86**	**0.139**	0.279	2.00

Fastest first canonicalization on every problem; re-solve within a factor of about two of the cached tensor.

On the last two, the data is almost entirely constant and the parameters are a few scalars — but a parameter update still forces the numeric pass to recompute the whole constant part.

Three observations

- **Caching does not require the DPP ruleset.** The symbolic and numeric split re-solves within a factor of about two of the cached tensor on every problem, while ignoring the ruleset entirely.
- **The tensor's upfront cost is not always justified.** The split's first canonicalization is faster than building the tensor on all seven problems, and its re-solve is faster on five of them.
- **`cvxcore` can be replaced.** `diffengine` canonicalizes faster on 24 of the 25 problems, and `COO` builds the tensor faster than `cvxcore` on all seven parametric ones.

Three observations

- **Caching does not require the DPP ruleset.** The symbolic and numeric split re-solves within a factor of about two of the cached tensor on every problem, while ignoring the ruleset entirely.
- **The tensor's upfront cost is not always justified.** The split's first canonicalization is faster than building the tensor on all seven problems, and its re-solve is faster on five of them.
- **`cvxcore` can be replaced.** `diffengine` canonicalizes faster on 24 of the 25 problems, and `COO` builds the tensor faster than `cvxcore` on all seven parametric ones.

and the downsides

- peak memory is higher on several problems: `SCIPY` is lowest on 16 of the 25, and `diffengine` uses about $1.4\times$ its memory
- the re-solve requires the cached program to stay in memory, where a built tensor does not
- the suite is a collection of synthetic problems, not a standard benchmark library

Future work

- a standalone `sparsediffpy` interface: build and evaluate sparse Jacobians and Hessians without `CVXPY`, with the trees built from `sparsediffpy`'s own functions
- `diffengine` as the default backend on the `ignore_dpp` path, and the removal of `cvxcore`, which the `CVXPY` developers have already deprecated — `CVXPY` would then ship no C++ binaries at all
- pass the block structure along with the problem data, so that the presolve and the factorization work on the dense blocks directly instead of rediscovering them in a generic sparse matrix

Thank you

Backup: the DCP rules

Every atom declares its curvature, its monotonicity in each argument, and the sign of its output:

Atom	Definition	Curvature	Monotonicity	Sign
`sum`	$\sum_i x_i$	affine	non-decreasing	any
`abs`	$\lvert x \rvert$	convex	unknown	nonnegative
`square`	x^2	convex	unknown	nonnegative
`sqrt`	$\sqrt{x}$	concave	non-decreasing	nonnegative
`norm`	$\lVert \mathbf{x} \rVert_2$	convex	unknown	nonnegative

The composition $\phi(u_1(\mathbf{x}), \ldots, u_k(\mathbf{x}))$ is convex when ϕ is convex and, for each argument i, one of the following holds:

- ϕ is non-decreasing in i and u_i is convex;
- ϕ is non-increasing in i and u_i is concave; or
- u_i is affine.

Swap convex and concave for the concave rule. A problem is DCP-compliant when the objective and every constraint come out compatible: convex objective, convex $\leq$ concave, affine equalities.

The rules are sufficient for convexity, not necessary: a genuinely convex problem written in an unrecognized form is rejected.

Backup: sufficient, not necessary

The same function, written two ways. Only one of them compiles.

$$\sqrt{x^2 + 1} \;=\; \left\|(x,\ 1)\right\|_2$$

```
cp.sqrt(cp.square(x) + 1)
```

rejected

- `sqrt` is concave and non-decreasing
- its argument $x^2 + 1$ is convex, not affine
- no rule covers concave $\circ$ convex, so the curvature is unknown

```
cp.norm(cp.hstack([x, 1]))
```

accepted

- `norm` is convex
- its argument $(x, 1)$ is affine
- convex $\circ$ affine is convex, and the epigraph is $(t,\ x,\ 1) \in \mathcal{K}_{\text{soc},3}$

The rules are a test on the expression, not on the function. Rewriting is the user's job; the atom library is what makes it possible.

Backup: disciplined parametrized programming

DPP is a second grammar, on top of DCP, restricting how parameters may enter an expression: every parameter is treated as affine, and products of two parametrized expressions are forbidden.

The payoff is that the problem data becomes affine in the parameters:

$$\mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}}, \qquad \operatorname{vec}([\mathbf{A}\ \mathbf{b}]) = \mathbf{R}\,\tilde{\boldsymbol{\theta}}, \qquad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} \boldsymbol{\theta} \\ 1 \end{bmatrix}$$

- `CVXPY` builds $\mathbf{Q}$ and $\mathbf{R}$ by adding a third dimension to every coefficient matrix of the pass, indexed by the parameter entries
- after a parameter change, a re-solve costs one sparse matrix-vector product per map: no tree is traversed again
- a problem outside the grammar, or one passing `ignore_dpp`, takes the `EvalParams` path instead: each parameter leaf is replaced by its current value and the whole canonicalization repeats

This is the baseline the parametric results are measured against.

Backup: two ways to get the coefficients

Every modelling language reviewed produces the same output, the solver problem data. They split into two families by what they ask of the user.

Differentiate an expression graph

- `CasADi`: built for nonlinear optimization; its conic interface `qpsol` differentiates the graph through matrix-vector products, so the sparsity pattern must be detected at runtime and compressed by colouring
- `CVXPY`: composes the linear operators of the conic-form trees in one bottom-up pass, which is equivalent to differentiating them; expressions stay vectorized, so no detection step is needed

Read the coefficients the user declares

- `AMPL`, `JuMP`: the user writes each objective and constraint as a sum of terms, and every linear and quadratic coefficient is stored as it is typed
- neither has an atom library or a canonicalization step, so a norm is left to the solver (`AMPL`) or written in conic form by hand (`JuMP`); both scalarize

Backup: the research gap

System	Expression storage	Coefficients obtained by	Reused across the family	Repeated at every solve
`AMPL`	scalar	instance generation	nothing, values folded into the instance	`.nl` write and read
`JuMP`	scalar	term lists stored by macros	term lists, modified in place	`copy_to` into the solver
`CasADi`	scalar or matrix	differentiation of an expression graph	pattern, colouring, derivative function	one sweep per colour
`CVXPY` backends	vectorized	composition of linear operators	parameter-to-data tensor, DPP problems only	one tensor-vector product, or the whole pass
`SparseDiffEngine`	vectorized	structured direct differentiation	problem structure, by the symbolic pass	numeric pass

None of the four combines the two-phase split with direct computation of the problem data.

Two phases like `CasADi`, but the whole coefficient matrix at once instead of matrix-vector products, and dense storage where the structure is dense.

Backup: ten kernels is the whole library

Kernel	Computes
`transpose`	$\mathbf{A}^\top$
`index`	$\mathbf{A}[\mathbf{r}, :]$
`sum_row_partition`	sums of groups of rows
`promote`	copies of a single row
`broadcast`	copies of a row or a column
`diag_vec`	rows moved to the diagonal
`DA`	$\mathrm{diag}(\mathbf{d})\,\mathbf{A}$
`sum_matrices`	$\mathbf{A} + \mathbf{B}$
`BA`	$\mathbf{BA}$
`BTA`, `BTDA`	$\mathbf{B}^\top\mathbf{A}$, $\mathbf{B}^\top\mathbf{DA}$

seven unary, three binary

- every atom of a canonicalized DCP problem reaches its derivative through one of these — there is no per-atom derivative code to write
- each kernel is written once per representation, and the call selects the implementation: the unary ones on their operand, the transposed products on both operands at once
- so dense and sparse mix freely: a sparse-dense pair is a kernel that converts neither operand, and a product with a dense operand gives back a dense one
- once a dense block appears, the dense path takes over and the structure is never flattened into (row, column, value) triples

Being free to pick the representation per operation is what the timings measure.

Backup: addition is where the dense path stops

- the Jacobian of $\mathbf{Bx} + \mathbf{y}$ holds the block of $\mathbf{B}$ at the columns of $\mathbf{x}$ *and* an identity at the columns of $\mathbf{y}$
- no single triple $(\mathbf{X}, \mathbf{r}, \mathbf{c})$ covers both, so the sum is performed on CSR views of the two operands instead
- the views are built during the symbolic pass and their value arrays point at the dense blocks, so no values are copied
- the result is a generic sparse matrix, and so is everything computed from it

The dense representations are sticky in the other direction: a product with a permuted dense or stacked operand gives back a matrix of the same kind.

Backup: what the two-phase split costs

- the numeric pass re-evaluates the expression trees at every parameter change, so the `problem` object must stay in memory for the lifetime of the problem family
- the parameter-to-data tensor has no such constraint: once it is built, a re-solve needs only its product with the current parameter values, and the formulation can be set aside
- a parameter update also forces the numeric pass to recompute the entire constant part of the problem data, which is why the two problems with almost-constant data and a few scalar parameters re-solve faster through the tensor

This is the one place the tensor is genuinely the better object.

Backup: how to read a performance profile

A performance profile compares the backends over the whole suite at once, rather than one problem at a time.

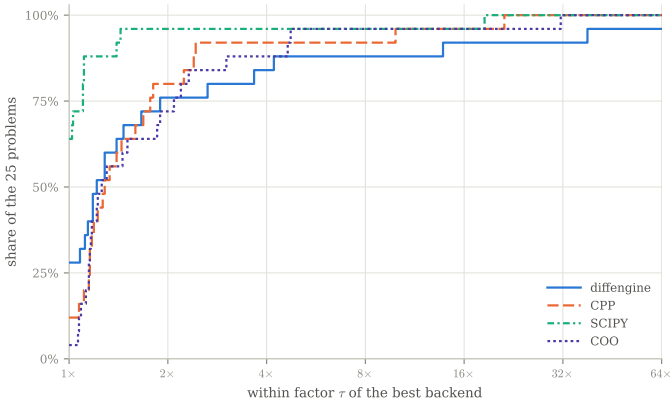
$$f_s(\tau) = \frac{1}{n} \left| \{ p : t_{p,s} \leq \tau \min_b t_{p,b} \} \right|$$

- $t_{p,s}$ is the time of backend s on problem p, and $\min_b t_{p,b}$ is the time of the fastest backend on that same problem
- $f_s(\tau)$ is the fraction of the n problems on which backend s lands within a factor τ of that best time
- at $\tau = 1$: the fraction of problems on which the backend is outright fastest
- where the curve reaches 1: the factor the backend needs in order to cover every problem in the suite

Dolan and Moré (2002).

Backup: peak memory

The same profile with $t_{p,s}$ replaced by the growth of the process's peak resident memory during canonicalization:



SCIPY has the lowest peak memory on 16 of the 25 problems.

diffengine matches CPP and COO in geometric mean, and uses about 1.4× the memory of SCIPY.